

Methodology: Multi-Day SOC-Aware EV Charging Simulation

Distribution-Grid-EV-CA/multiday_charging/

Replication documentation

June 4, 2026

Abstract

This document describes the `multiday_charging` subproject: a stand-alone Python simulation that samples EV charging sessions over many days using battery state-of-charge (SOC) logic, configurable charging-frequency preferences, and an empirical session pool. It complements the main grid pipeline documented in `pipeline_methodology.tex`, which builds charging *events* from CSTDM trips (SDPTM, LDPTM, ETM) and only attaches empirical start times and power in stage 05.

Contents

1	Purpose and relation to the main pipeline	3
2	Software layout	3
3	Required inputs	3
3.1	Empirical session pool (required)	3
3.2	Optional SDPTM calibration	3
4	Configuration (MultiDayConfig)	4
4.1	Fleet and energy	4
4.2	Charging-frequency matrix	4
4.3	Charging-location matrix	4
4.4	Sub-type weights (within location)	4
4.5	Daily driving and variability parameters	4
4.6	Parameter provenance (quick reference)	5
5	Simulation algorithm	5
5.1	Step 1: Build fleet (<code>fleet.py</code>)	5
5.2	Step 2: Daily inputs (<code>trips.py</code>)	5
5.3	Step 3: Session pool (<code>session_pool.py</code>)	5
5.4	Step 4: Multi-day sampling (<code>sampling.py</code>)	5
5.4.1	TAZ-level recharge energy (<code>result.taz_demand_kwh</code>)	5
5.4.2	Matching demand kWh to empirical session kWh	6
5.5	Step 5: Load profiles (<code>load_profile.py</code>)	6
6	Outputs and example figures	6
6.1	Files written by the notebook	6
6.2	Example run (default notebook configuration)	7
6.3	TAZ-level recharge energy maps	7
6.4	Daily curve variability vs. <code>Reference_Code/6</code>	7

7	How the main pipeline samples SD, LD, and ETM charging	7
7.1	Short distance (SDPTM) — 03_01	10
7.2	Long distance (LDPTM) and external (ETM) — 03_02	10
7.2.1	What TAZfreq means	10
7.2.2	Option 4: which TAZ gets a charging event	10
7.2.3	Consistency with Yanning's R script	11
7.3	Where empirical sessions attach — 05_02	12
8	Run commands	12

1 Purpose and relation to the main pipeline

The main pipeline (01_01–07_05) answers: *where and how many* charging events occur on feeders over adoption years, then copies *when and how fast* from real charger logs. The `multiday_charging` subproject answers a different question: *given a fleet with heterogeneous batteries and driving, when do vehicles choose to charge, and what hourly load shapes emerge over 365 days?*

Aspect	Main pipeline (03–05)	<code>multiday_charging</code>
Trip source	CSTDM SD/LD/ETM tables	Synthetic daily VMT + optional SDPTM calibration
Charge timing	Per trip / per corridor stop	SOC + preferred interval (1–7 days)
Empirical data	Stage 05 only (05_02)	Required pool at build time
Spatial output	Block / feeder	Fleet-aggregate hourly (optional zone column)

2 Software layout

```
multiday_charging/  
  config.py # MultiDayConfig dataclass (all parameters)  
  fleet.py # Battery mix, access flags, interval preference  
  trips.py # Daily VMT + work/public availability  
  session_pool.py # Load charging_session_all_clean.pkl  
  choice.py # Location / DC-L2 / housing weights  
  sampling.py # SOC-aware multi-day sampler  
  load_profile.py # Hourly matrices and percentile bands  
  run_multiday_charging.ipynb  
  outputs/ # Plots and arrays (created at run time)
```

Parent helpers: `../common.py` (`read_rds_like`, `save_rds_like`). Input builder: `../build_multiday_inputs.py`

3 Required inputs

3.1 Empirical session pool (required)

Path: `data/charging_data/charging_session_all_clean.pkl`

Built by `build_multiday_inputs.py`:

- **Home** — PEV-Profiles L1/L2 (PEV-Profiles-L1.xlsx, PEV-Profiles-L2.xlsx): 10-minute power profiles converted to sessions.
- **Work / public** — `intermediate_geolabeled_with_density_category 2.csv`: non-residential sessions (property-type mapping follows `Reference_Code/6_charging_behavior_synthetic_L1`)

Columns used: `charge_type` (home, work, public), `charger_level` (L2, DC), `housing` (home only), `energy` (kWh), `power` (kW), `start_hour`, `end_hour`. Bins 1–8 match stage 05: $(0, 5], \dots, (80, \infty]$ kWh.

3.2 Optional SDPTM calibration

Path: `data/mobility_data/CSTDM_processed/EV_trips_new_SDPTM_sample42.pkl`

When `CALIBRATE_FROM_REAL_TRIPS=True` in the notebook:

- Per-vehicle `mean_daily_vmt` $\leftarrow \sum \text{Dist by SerialNo.}$
- `work_access` $\leftarrow \text{configured share} \wedge \text{vehicle ever has ChargeType=W.}$

Calibration uses a lightweight profiles file (`EV_trips_new_SDPTM_sample42_profiles.pkl`) built by streaming SDPTM CSVs when the full trip pickle is too large for memory. Day-to-day VMT noise still uses `vmt_day_to_day_cv`; consider filtering trips to one year if multi-year inflation is unwanted.

4 Configuration (MultiDayConfig)

All parameters are set in one notebook cell (defaults in `config.py`).

4.1 Fleet and energy

- `battery_mix` — {kWh: share} capacity distribution.
- `efficiency_mi_per_kwh` = 3.0 — matches parent rule `demand = dist/3`.
- `reserve_soc` — minimum pack fraction retained; usable energy = `battery_kwh(1 - reserve_soc)`.

4.2 Charging-frequency matrix

`charge_interval_probs`: map $\{1, \dots, 7\} \rightarrow$ probability. Each vehicle draws `interval_pref_days`. Charging may occur earlier if the pack would deplete.

4.3 Charging-location matrix

`location_weights` default aligns with Reference_Code RAW_SHARES:

Label	Weight
Home (<code>Residential_L1_L2</code>)	0.68
Work (<code>Workplace_L2</code>)	0.04
Public (<code>Public_L2 + DCFC</code>)	0.28

Feasibility: `home_access_share`, `work_access_share`, `daily_work_trip_prob`, `public_trip_prob`. Infeasible locations get zero weight; remaining weights are renormalised (public is the backstop).

4.4 Sub-type weights (within location)

- `public_level_weights` — DC vs L2 when sampling public (default 0.20 / 0.08, normalised).
- `home_housing_weights` — single vs multi family (56.4 / 38.9, same as 05_02).

4.5 Daily driving and variability parameters

Mean VMT: `mean_daily_vmt` (default 32–35 mi) is a planning prior, close to `AVG_DAILY_VMT_PER_EV` = 35 in `Reference_Code/6_charging_behavior_synthetic_load.ipynb`. With SDPTM calibration, each vehicle’s personal mean is overwritten by $\sum \text{Dist}$ per `SerialNo` (optionally filter to one year to avoid multi-year inflation).

Coefficient of variation (CV): `vmt_between_vehicle_cv` and `vmt_day_to_day_cv` are *not* probabilities in $[0, 1]$. They are dimensionless spreads $CV = \sigma/\mu$:

- `vmt_between_vehicle_cv` (e.g. 0.45) — spread of `mean_daily_vmt` across vehicles via a lognormal draw in `fleet.py`.
- `vmt_day_to_day_cv` (e.g. 0.55) — day-to-day multiplier $\varepsilon_{v,d}$ around each vehicle’s mean in `trips.py`, with $\text{median}(\varepsilon) = 1$.

Implementation: $\sigma = \sqrt{\log(1 + CV^2)}$ for the underlying lognormal so the target CV is preserved.

Other: `no_travel_prob` (e.g. 0.08) is a true probability — fraction of days with zero miles.

4.6 Parameter provenance (quick reference)

Parameter	Typical source
public_level_weights 8919 / 29229	05_02 public charger counts (DC vs L2)
home_housing_weights 56.4 / 38.9	05_02 housing-unit weights
location_weights 0.68 / 0.04 / 0.28	RAW_SHARES in reference notebook
home_access_share, work_trip_prob, ...	Engineering defaults for feasibility
charge_interval_probs	User-tunable; not from CSTDM

5 Simulation algorithm

5.1 Step 1: Build fleet (fleet.py)

Draw per vehicle: `battery_kwh`, `interval_pref_days`, `home_access`, `work_access`, `mean_daily_vmt`. The notebook then calls `trips.assign_home_taz`: each vehicle draws a home TAZ (HomeZone) from `EVhh_new_SDPTM_sample42.csv` with probability proportional to the EV-household count per TAZ.

5.2 Step 2: Daily inputs (trips.py)

For each (vehicle, day):

$$\text{vmt}_{v,d} = \text{mean_daily_vmt}_v \times \varepsilon_{v,d}, \quad \varepsilon \sim \text{Lognormal}, \text{ median}(\varepsilon) = 1 \quad (1)$$

Zero out VMT with probability `no_travel_prob`. Set `work_available` and `public_available` from Bernoulli draws and access flags.

5.3 Step 3: Session pool (session_pool.py)

Load empirical pool; assign bin from `energy`; assign `eventID`.

5.4 Step 4: Multi-day sampling (sampling.py)

For each day d and vehicle v :

1. Energy consumed: $e_{v,d} = \text{vmt}_{v,d} / \text{efficiency_mi_per_kwh}$.
2. Accumulate $E_v^{\text{cum}} += e_{v,d}$; increment `days_since`.
3. **Charge if** $E_v^{\text{cum}} \geq \text{usable_kwh}_v$ **or** `days_since` $\geq \text{interval_pref_days}_v$ (and VMT > 0 that day).
4. Feasible locations: home if `home_access`; work if `work_available`; public if `public_available`.
5. Draw location from renormalised `location_weights`; draw housing or DC/L2 sub-type.
6. Compute recharge **demand** $= \min(E_v^{\text{cum}}, \text{usable_kwh}_v)$; map demand to energy bin b (same cutpoints as stage 05).
7. Sample one empirical row with matching (`charge_type`, sub-type, b); copy `start_hour`, `end_hour`, `power`, and pool `energy` as `energy_session`.
8. Reset E_v^{cum} and `days_since` (full recharge assumption).

5.4.1 TAZ-level recharge energy (result.taz_demand_kwh)

When the fleet has `home_taz`, the sampler accumulates annual recharge energy (kWh) by TAZ and location type:

- **Home** — credited to the vehicle's assigned `home_taz`.

- **Work / public** — credited to a TAZ drawn at random with weights equal to the fleet’s home-TAZ distribution (a destination proxy when trip-level destination TAZ is not carried through the multi-day loop).

The notebook exports `taz_total_demand_kwh.csv` (columns TAZ, home_kwh, work_kwh, public_kwh, total_kwh) and joins these totals to CSTDM polygons `data/shps/taz_id.shp` (field TAZ12) for choropleth maps.

5.4.2 Matching demand kWh to empirical session kWh

The simulation stores two energy fields per sampled session:

- **energy_demand** — kWh the vehicle needed since the last charge (from accumulated driving, capped by usable pack).
- **energy_session** — kWh from the *drawn* historical session in the pool.

They are **not forced to be equal**. Matching is **stratified by bin only**:

1. Driving sets **energy_demand** (e.g. 9.3 kWh).
2. Demand maps to bin b (e.g. bin 2: (5, 10] kWh).
3. A random pool row is drawn with the same **charge_type**, housing or DC/L2 sub-type, and bin b (e.g. 8.2 kWh).

This mirrors stage 05_02: synthetic events get $\text{demand} = \text{dist}/3$ and a bin; empirical `eventID` supplies timing and power without rescaling session energy to synthetic demand.

What drives hourly load plots: `load_profile.py` uses empirical **power** (kW) and `start_hour/end_hour` only — not **energy_demand**. Integrated energy over active hours tracks **energy_session**, which is only loosely tied to demand through the shared bin. The design is bootstrap / stratified re-sampling: SOC logic sets *when* and *how large* (bin); the pool sets *time-of-day shape* and charger power.

5.5 Step 5: Load profiles (load_profile.py)

Build $[n_{\text{days}}, 24]$ matrix: constant power over session hours (wrap at midnight). Outputs: daily curves, 5th–95th hourly percentiles, optional split by **charge_type**.

6 Outputs and example figures

Default run directory: `multiday_charging/outputs/`. The figures in Section 6.2 are taken directly from that folder after running `run_multiday_charging.ipynb`.

6.1 Files written by the notebook

- `daily_load_365_lines.png`
- `hourly_load_percentile_band.png`
- `mean_load_by_type.png`
- `daily_hourly_load_kW.npy`
- `hourly_percentile_band.csv`
- `taz_total_demand_kwh.csv`
- `taz_total_demand_map.png`
- `taz_demand_by_type_maps.png` (home / work / public panel)
- `taz_home_demand_map.png`, `taz_work_demand_map.png`, `taz_public_demand_map.png`
- `sampled_sessions.parquet` (when `store_sessions=True`)

For full California fleet scale ($n_{\text{vehicles}} \approx 2.5 \times 10^6$), set `store_sessions=False` so the sampler builds hourly load matrices only (no multi-gigabyte session table).

6.2 Example run (default notebook configuration)

The figures below use `seed=42`, `n_vehicles=2,500,000`, `n_days=365`, SDPTM profile calibration, and the empirical session pool. Hourly load is total fleet kW summed over active session hours (constant `power` per session, same convention as stage 06).

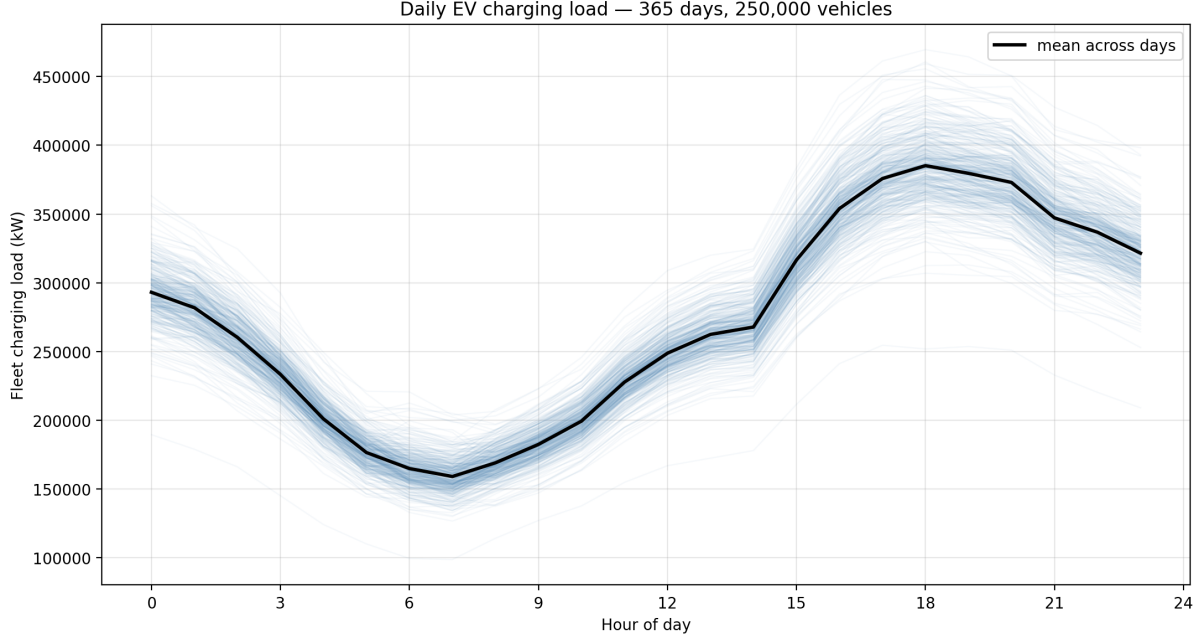


Figure 1: All 365 daily fleet charging load curves (light blue) and the mean curve across days (black). At large fleet size, curves often appear nearly parallel because the law of large numbers stabilises the hour-of-day *shape*; day-to-day differences are mostly small changes in total scale.

6.3 TAZ-level recharge energy maps

Annual recharge energy (GWh) by CSTDM TAZ, from `result.taz_demand_kwh`. Grey polygons have zero simulated demand for that layer. Color scales are logarithmic (5th–99th percentile of positive TAZs per layer). Home energy concentrates in residential TAZs where EV households are sampled; work and public layers use the fleet home-TAZ weight proxy described in Section 5.4.1.

6.4 Daily curve variability vs. Reference_Code/6

The reference synthetic-load notebook (`6_charging_behavior_synthetic_load.ipynb`) intentionally widens day-to-day *shape* variance: residential days resample whole fleet-day shapes from PEV profiles; public/work pools use a large `SUPER_SESSION_FACTOR` so each day draws a small random set of session tuples. `multiday_charging` instead draws one empirical session per charging *event*; at millions of vehicles, independent draws average to a stable aggregate profile (Figure 1). To increase visible spread: spread `charge_interval_probs` across 2–7 days, raise `vmt_day_to_day_cv` or `no_travel_prob`, or plot normalised daily shapes / percentile bands rather than raw kW overlays.

7 How the main pipeline samples SD, LD, and ETM charging

This section documents stages 03 and 05 of the parent pipeline. `multiday_charging` does not run these steps; it is included so both documents stay self-contained.

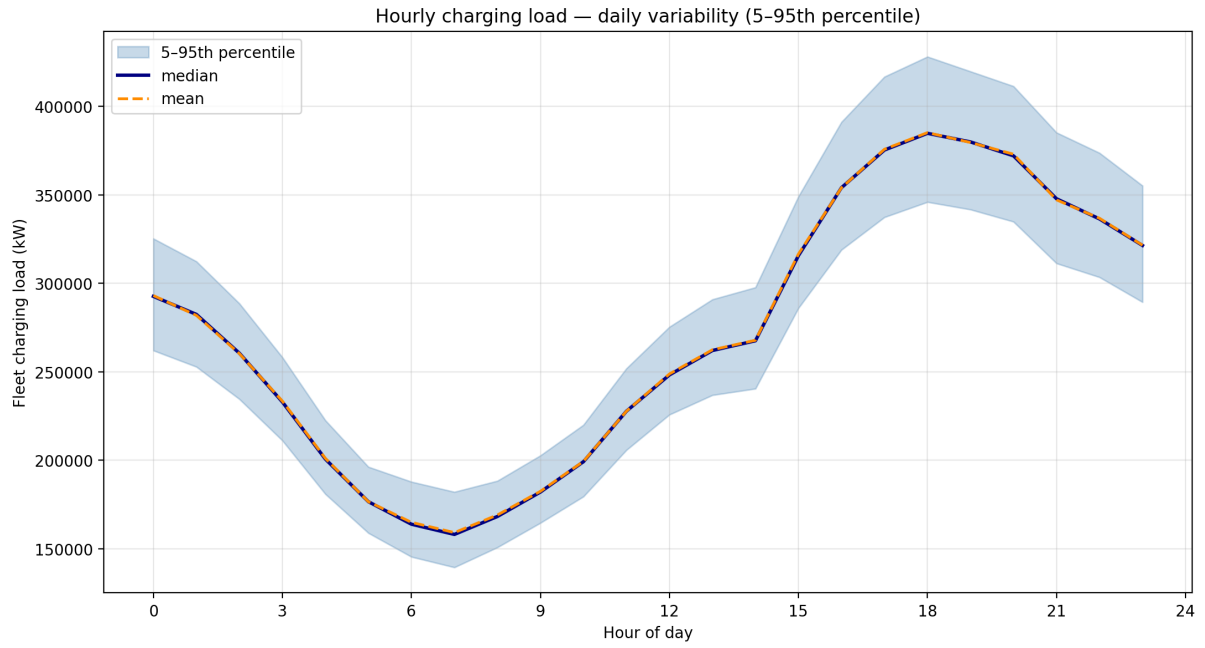


Figure 2: Hourly load variability across the 365 simulated days: 5th–95th percentile band (shaded), median (navy), and mean (dashed orange). This view highlights residual day-to-day spread that is hard to see when overlaying all daily curves.

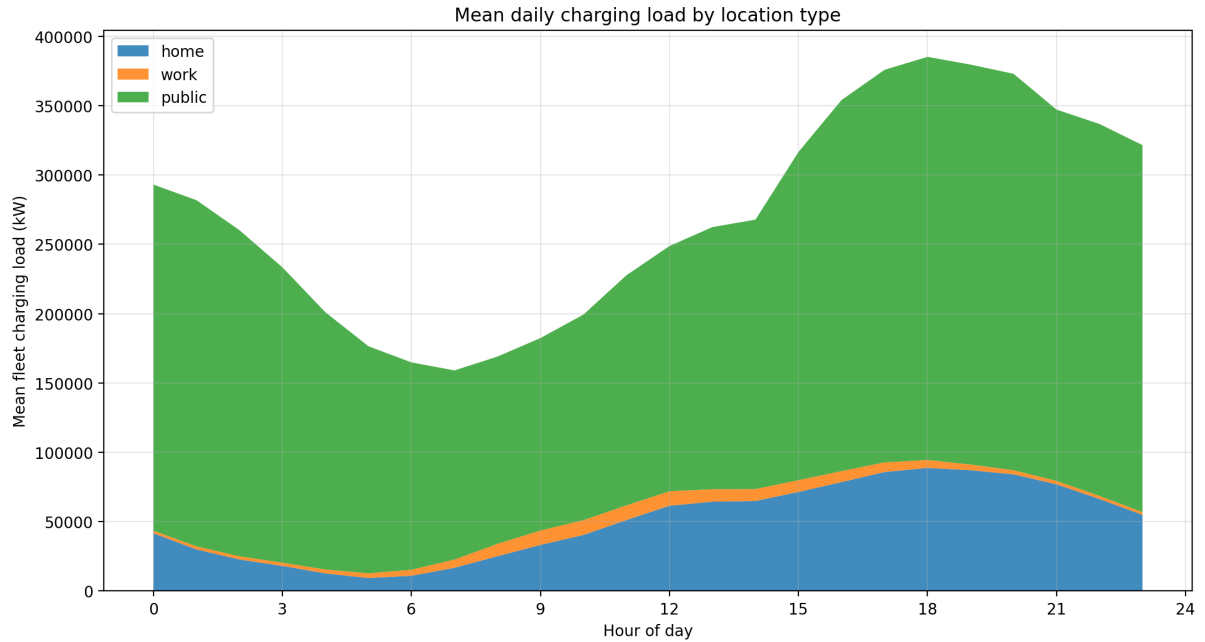


Figure 3: Mean daily charging load decomposed by location type (home, work, public), stacked by hour. Home dominates under default `RAW_SHARES` location weights (68% / 4% / 28%).

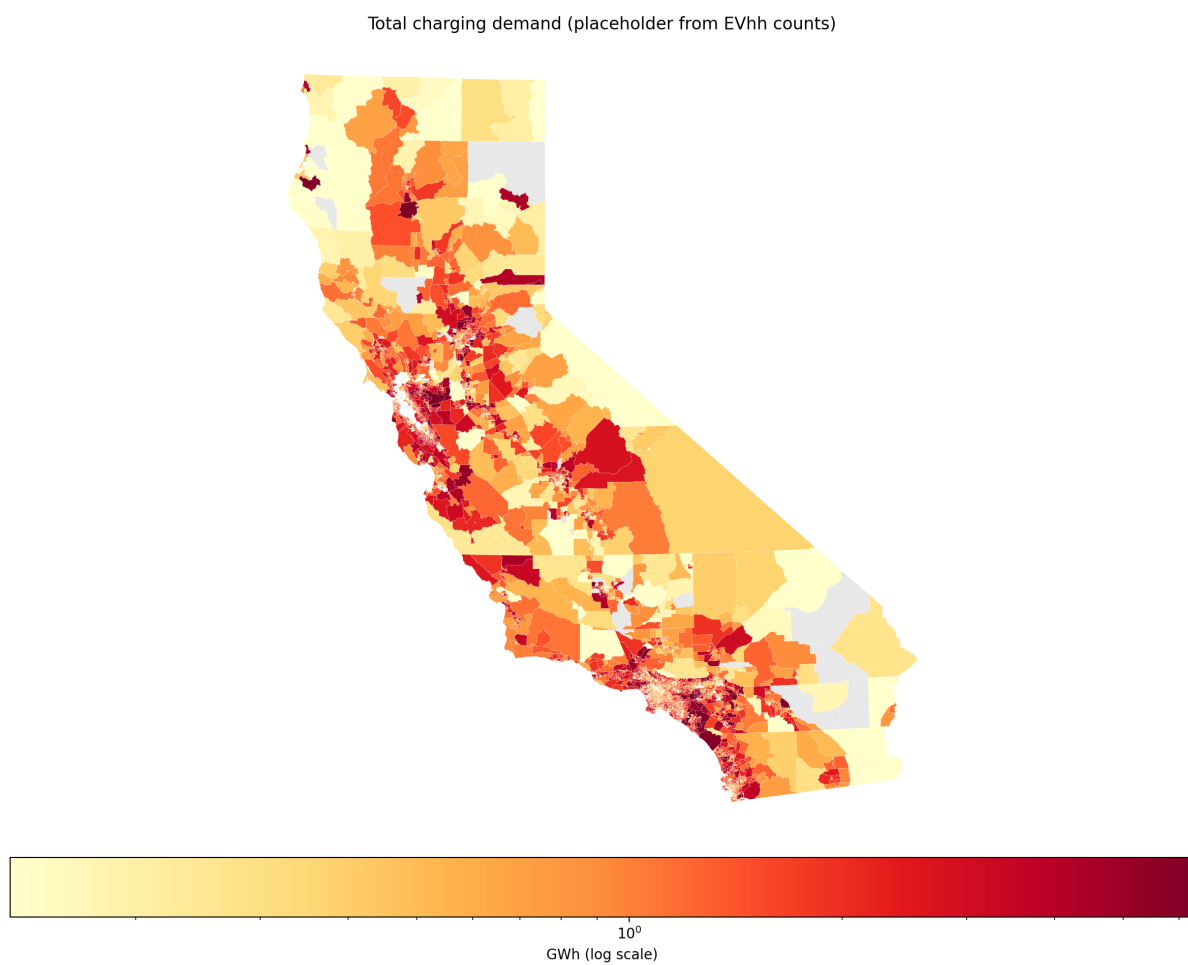


Figure 4: Total annual recharge energy by TAZ (home + work + public).

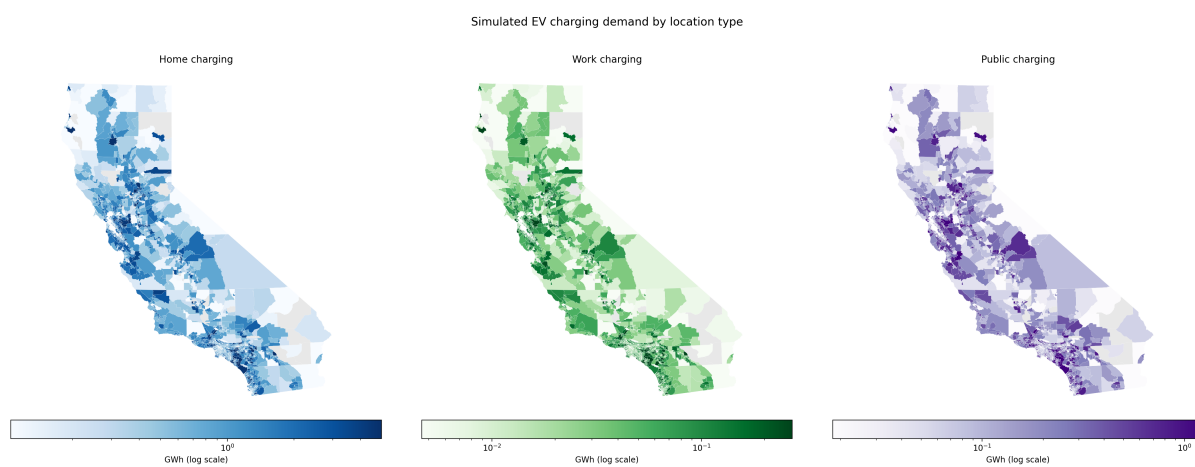


Figure 5: Annual recharge energy by TAZ and location type: home (left), work (center), public (right).

7.1 Short distance (SDPTM) — 03_01

Input: EV trips_new_SDPTM_sample42 with ChargeType $\in \{H, W, P\}$ from trip purpose.

Logic:

1. Group trips into tours (SerialNo, Person, Tour).
2. For each tour, list which of $\{H, W, P\}$ appear among trips.
3. Form seven valid location *patterns* (e.g. home-only, home+work, all three) with EVMT survey shares (53% home-only, 8% work-only, ...).
4. Normalise shares to the tour; draw one pattern (seed 12).
5. A trip creates a charging event if its ChargeType matches an active location in the drawn pattern.
6. Sum Dist over trips assigned to each event $\rightarrow$ chargeDist; attach TAZ J, Time, year.

Output: charge_event_new_SDPTM_sample42_draw12. No empirical session times yet — only *that* charging happens, *where* (H/W/P), and *how far* (miles).

7.2 Long distance (LDPTM) and external (ETM) — 03_02

Reference: R_Yanning/03_02_charging_events_LD_ETM.R. Options 1–3 are explored in R but **not saved**; both the R workflow and the Python port persist **Option 4 only** to charge_event_new_LDPTM_samp and charge_event_new_ETM_sample42.

Inputs:

- EV trips (EV trips_new_LDPTM_sample42 or EV trips_new_EXT_sample42).
- Parsed skim: parsed_100000_LDPTM or parsed_100000_ETM — one row per TAZ on the shortest-path sequence for each (I, J) .

7.2.1 What TAZfreq means

On the *full* skim table (all modeled long-distance paths, before joining EV trips), Yanning defines:

```
dist.ld[, TAZfreq := .N, by = TAZway] # R data.table
```

TAZfreq for TAZ k = number of skim rows where TAZway = k . Because each origin–destination pair has many rows (one per TAZ along the path), a high TAZfreq means that TAZ appears on many $I \rightarrow J$ routes in the LD (or ETM) network — a proxy for a **major corridor** / **hub**, not census population, charger inventory, or EV trip counts.

What it is not: EV trip frequency, distance since last charge, or geographic area.

7.2.2 Option 4: which TAZ gets a charging event

Option 4 does **not** place a charger every 100 miles along the odometer. It selects TAZs **on the trip's path** that are also **high-TAZfreq** relative to other TAZs on that same path, targeting about $\lceil D/100 \rceil$ stops.

Step-by-step (per trip).

1. **Target stop count:** $n_{\text{charge}} = \lceil D/100 \rceil$ (e.g. 220 mi $\Rightarrow$ 3 stops).
2. **Path length:** $n_{\text{TAZ}, \text{trip}}$ = number of skim rows for this TripID on (I, J) .
3. **Quantile probability:**

$$q_{\text{cut}} = 1 - \frac{n_{\text{charge}}}{n_{\text{TAZ}, \text{trip}}} \quad (2)$$

Many path TAZs but few required stops $\Rightarrow$ high $q_{\text{cut}} \Rightarrow$ stricter filter.

4. **Cutoff on this path only:** Let $F = \{\text{TAZfreq of TAZs on this trip's path}\}$. Then

$$\text{freqCUT} = Q_{q_{\text{cut}}}(F) \quad (3)$$

(R: `quantile(TAZfreq, probs=qtCUT, by=TripID)`; Python: `np.quantile` per trip.)

5. **Keep TAZs:**

- LDPTM: `TAZfreq > freqCUT`
- ETM: `TAZfreq >= freqCUT` (R line 160; slightly looser than LD)

6. **Attributed miles per stop:** $\text{chargeDist} = D/n_{\text{TAZ,charge}}$ where $n_{\text{TAZ,charge}}$ is the count of kept TAZs. Distance is split **evenly** across selected stops, not by actual spacing along the path.

LD labeling after TAZ selection.

- Corridor TAZs (`TAZway ≠ J`): `ChargeEventType = P` (public), `other = corridor`.
- Destination (`TAZway = J`): `ChargeEventType = trip ChargeType` (H/W/P), `other = destination`.

ETM. Same Option 4 on `parsed_100000_ETM`; trips are inbound to California; output keeps `ChargeType` (typically public). No `ChargeEventType/other` corridor split in the saved ETM file.

Numeric example. Trip 220 mi, 40 TAZ rows on path: $n_{\text{charge}} = 3$, $q_{\text{cut}} = 1 - 3/40 = 0.925$. If `TAZfreq` on those 40 TAZs ranges 50–5000, `freqCUT` might be ≈ 4200 . Kept TAZs have `TAZfreq > 4200` — often ~ 3 busy highway TAZs, not necessarily spaced every 73 mi. Each gets $\text{chargeDist} \approx 220/3 \approx 73$ mi.

What Option 4 is not.

- Not all TAZs on the path (Option 1).
- Not a fixed global top-10% of TAZs (Option 3 uses 90% quantile fixed).
- Not geometric “every 100 miles” along the route polyline.

7.2.3 Consistency with Yanning’s R script

The Python port `03_02_charging_events_LD_ETM.py` matches the **saved** R Option 4 logic:

Step	R (<code>03_02_charging_events_LD_ETM.R</code>)
<code>TAZfreq</code>	<code>.N, by=TAZway</code> on full <code>dist.ld / dist.etm</code>
n_{charge}	<code>ceiling(Dist/100)</code>
q_{cut}	<code>1 - nCharge/nTAZtrip</code>
<code>freqCUT</code>	<code>quantile(TAZfreq, probs=qtCUT, by=TripID)</code>
LD filter	<code>TAZfreq > freqCUT</code> (line 88)
ETM filter	<code>TAZfreq >= freqCUT</code> (line 160)
<code>chargeDist</code>	<code>Dist/nTAZcharge</code>

Minor implementation notes: R `quantile` vs NumPy may differ slightly at bin edges; ETM trip–path join in R is trip-driven (`allow.cartesian=TRUE`) whereas Python uses a left merge on `dist` — results align when every ETM trip has a skim match.

7.3 Where empirical sessions attach — 05_02

Stages 03–04 produce **synthetic events**: which TAZ or block, how many events, `chargeDist` or `dist` (miles). Energy need for binning:

$$\text{demand (kWh)} = \frac{\text{dist}}{3} \quad (4)$$

Eight bins $(0, 5], \dots, (80, \infty]$ on both synthetic demand and pool **energy**.

Stage 05_02 then:

1. Assigns public DC vs L2 (charger counts 8919 / 29229) and home housing (56.4 / 38.9); LD corridor trips often forced to DC.
2. For each stratum (type, housing or level, bin), draws `eventID` from `charging_session_all_clean` (seed 36).
3. Copies `start_hour`, `end_hour`, `power`, `energy` — same stratified bootstrap principle as `multiday_charging`, but spatially at feeder/block level after stage 04.

Stage 06 expands each event to hourly feeder load using constant **power** over `[start_hour, end_hour]`.

8 Run commands

```
# Build inputs (once)
python build_multiday_inputs.py
python build_multiday_inputs.py --profiles-only # SDPTM profiles only

# Interactive run (writes figures to multiday_charging/outputs/)
jupyter notebook multiday_charging/run_multiday_charging.ipynb

# Compile this report (from Distribution-Grid-EV-CA/docs/)
pdflatex multiday_charging_methodology.tex
pdflatex multiday_charging_methodology.tex
```